Warsaw University of Technology

FACULTY OF
ELECTRONICS AND INFORMATION TECHNOLOGY

Institute of Control and Computation Engineering

# Bachelor's diploma thesis

in the field of study Computer Science
and specialisation Software Engineering

Web/desktop application (TypeScript) for visualizing and composing dependency graphs between reusable software components

Binh Vuong Le Duc
student record book number 330097

thesis supervisor
mgr inż. Sławomir Niespodziany

WARSAW 2026

## Web/desktop application (TypeScript) for visualizing and composing dependency graphs between reusable software components

**Abstract.** Modern embedded systems development increasingly relies on modular software architectures to manage complexity and enhance reusability. The `diff` framework, a custom-built dependency injection system for resource-constrained C++ environments, enables dynamic system configuration through JSON-based topology files. However, as the number of interconnected components grows, manual maintenance of these configuration files becomes error-prone and lacks intuitive structural oversight.

This thesis presents the design and implementation of `diff-forge`, a desktop application developed with TypeScript, React, and Electron, specifically tailored for the visual composition of software topologies within the `diff` ecosystem. The proposed solution provides a graphical canvas for managing component instances and their dependencies, complemented by a real-time validation engine for cycle and orphan detection. By bridging the gap between abstract topology definitions and visual representation, `diff-forge` aims to improve developer productivity and reduce configuration errors in the assembly of complex embedded software systems.

**Keywords:** TypeScript, Electron, Dependency Injection, Visual Programming, Embedded Systems

# Contents

# 1. Introduction

## 1.1. Motivation

The rapid advancement of semiconductor technology and the increasing demand for sophisticated functionality in embedded systems have shifted the primary challenge of development from hardware constraints to software complexity [1]. As embedded devices integrate into critical infrastructure, such as automotive control units and industrial IoT gateways, the need for scalable and maintainable software architectures has never been greater.

In traditional embedded development, software components often exhibit coupling with the underlying hardware, hindering reusability across different projects. To address this, modular frameworks like `diff` utilize the Dependency Injection (DI) pattern [2] to decouple component logic from system configuration. While this approach improves testability and flexibility, it introduces a new layer of complexity: the management of the dependency graph itself.

In the current workflow, developers must manually define the system topology in structured JSON files. While functional, this "blind wiring" process is inherently error-prone. As system complexity increases, detecting circular dependencies, identifying unfulfilled requirements, or understanding the architecture from a text-based file becomes a significant cognitive burden. A specialized tool that provides visual feedback and automated verification during the system composition phase is essential.

## 1.2. Problem Statement

The core problem addressed by this thesis is the lack of a visual management layer for the `diff` dependency injection framework. Specifically:

- **High Cognitive Load:** Managing hundreds of component connections in a JSON format is difficult to audit and visualize.
- **Delayed Error Detection:** Structural issues, such as dependency cycles, are often only caught at runtime, leading to expensive debugging cycles.
- **Configuration Fragility:** Manually editing IDs and dependency arrays is susceptible to typos and reference errors that a standard text editor cannot verify.

## 1.3. Project Goals and Scope

The primary goal of this project is to develop `diff-forge`, a cross-platform desktop application that enables developers to visually compose, verify, and export software topologies. The application utilizes modern web technologies including TypeScript, React [3], and Electron [4]. The specific goals of the application include:

- Providing an intuitive drag-and-drop interface for instantiating components from a managed catalog.
- Implementing a real-time validation engine to prevent structural errors before they reach the production environment.

- Ensuring seamless integration with existing C++ development workflows by exporting framework-compliant configuration files.

### 1.4. Thesis Structure

The following chapters comprise this thesis: Chapter 2 reviews the state of the art in dependency injection and visual programming. Chapter 3 provides an overview of the `diff` framework and its target use cases. Chapter 4 describes the architectural design of `diff-forge`, while Chapter 5 details its implementation. Lastly, Chapter 6 evaluates the tool's effectiveness, followed by concluding remarks in Chapter 7.

# References


[1] S. Niespodziany, “Data-driven dependency injection for embedded software, enhancing reusability with c++”, *International Journal of Electronics and Telecommunications*, vol. 71, no. 4, pp. 1–7, 2025. DOI: 10.24425/ijet.2025.155469

[2] M. Fowler, *Inversion of Control Containers and the Dependency Injection pattern*. martinfowler.com, 2004. [Online]. Available: https://martinfowler.com/articles/injection.html

[3] I. Meta Platforms. “React documentation”. [Online]. Available: https://react.dev/

[4] O. Foundation. “Electron documentation”. [Online]. Available: https://www.electronjs.org/docs/latest

# List of Symbols and Abbreviations

**EiTI** – Faculty of Electronics and Information Technology
**WUT** – Warsaw University of Technology
**DI** – Dependency Injection
**IAiIS** – Institute of Control and Computation Engineering
**CBSE** – Component-Based Software Engineering
**GUI** – Graphical User Interface

# List of Figures

# List of Tables

# List of Appendices

# Appendix 1. User Manual

Instructions for installing and running `diff-forge`.

# Appendix 2. Configuration Schema

The JSON schema used for component metadata.